Fiche de Révision Complète

Reinforcement Learning — EFREI 2024/2025

Toutes les notions classées par priorité, avec formules et exemples d'examen

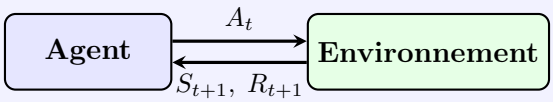
Guide de lecture – Les TIERs

- **TIER S** : Apprendre EN PREMIER. Quasi-certain de tomber. Facile, gros gain de points. (∼20 pts)
- **TIER A** : Apprendre EN SECOND. Très probable. Définitions courtes, 1pt chacune. (∼10 pts)
- **TIER B** : Apprendre ENSUITE. Probable. Demande un peu plus de compréhension. (∼10 pts)
- **TIER C** : Apprendre SI TEMPS RESTANT. Possible ou bonus. (∼6 pts + bonus)

TIER S — PRIORITÉ ABSOLUE (∼20 pts, 30 min d'apprentissage)

1 S1. Le schéma général du RL (3 pts)

La boucle Agent–Environnement



Boucle à chaque pas de temps t :

1. L'agent observe l'état S_t et reçoit la récompense R_t
2. L'agent choisit et exécute l'action A_t
3. L'environnement produit le nouvel état S_{t+1} et la récompense R_{t+1}
4. $t \leftarrow t + 1$, on recommence

Les 5 variables à définir

Variable	Définition
Agent	Système qui prend des décisions séquentielles pour maximiser ses récompenses cumulées
Environnement	Système externe avec lequel l'agent interagit ; produit états et récompenses
État S_t	Description complète de la situation actuelle de l'environnement
Action A_t	Décision ou mouvement effectué par l'agent dans un état donné
Récompense R_t	Signal numérique de feedback évaluant la qualité de l'action

Analogie – Dressage d'un chien

Chien = **Agent** | Monde = **Environnement** | Ce qu'il voit = **État**
Ce qu'il fait (s'asseoir) = **Action** | Biscuit ou "non !" = **Récompense**
Le chien apprend à maximiser ses biscuits = c'est exactement le RL.

Question type examen (Q1, 3 pts)

"Schématisez le cadre général du RL. Nommez et définissez les variables."

Ce qu'on attend : le schéma Agent ↔ Environnement avec les flèches A_t, S_{t+1}, R_{t+1}, puis le tableau des 5 définitions.

2 S2. Définition d'un MDP (1 pt)

MDP = Markov Decision Process

Un MDP est un tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ où :
- $\mathcal{S}$: ensemble (fini) d'états
- $\mathcal{A}$: ensemble (fini) d'actions
- P : probabilités de transition $\rightarrow$ $p(s'|s,a)$
- R : fonction de récompense $\rightarrow$ $p(r|s,a)$
- γ : facteur d'actualisation (discount factor)

Propriété de Markov :

$$P(S_{t+1} \mid S_t, A_t) = P(S_{t+1} \mid S_0, S_1, \ldots, S_t, A_t)$$

L'état futur ne dépend **que** de l'état présent et de l'action, **pas** de l'historique.

Analogie – Jeu d'échecs

$\mathcal{S}$ = toutes les positions du plateau, $\mathcal{A}$ = coups légaux, $R = +1$ victoire / -1 défaite.
Propriété de Markov : pour jouer, il suffit de voir le plateau actuel, pas comment on y est arrivé.

Question type examen (Q2, 1 pt)

"Qu'est-ce qu'un MDP et comment le définit-on ?"

Réponse : donner le tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ + propriété de Markov.

3 S3. Les 3 définitions : V, Q, G (3 pts)

Gain G – cumulative discounted reward

$$\boxed{G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k} = r_t + \gamma \, r_{t+1} + \gamma^2 \, r_{t+2} + \cdots}$$

Forme récursive : $G_t = r_t + \gamma \, G_{t+1}$

Value Function $V^\pi(s)$ – state-value

$$\boxed{V^\pi(s) = \mathbb{E}[G_t \mid S_t = s]}$$

L'espérance du gain en partant de l'état s, en suivant la politique π.

Action-Value Function $Q^\pi(s,a)$

$$\boxed{Q^\pi(s,a) = \mathbb{E}[G_t \mid S_t = s,\ A_t = a]}$$

L'espérance du gain en partant de s, en prenant l'action a, puis en suivant π.

Liens entre V et Q

$$V^\pi(s) = \sum_a \pi(a|s)\, Q^\pi(s,a) \qquad V = \text{moyenne de } Q \text{ sur les actions}$$

$$Q^\pi(s,a) = \mathbb{E}\big[r + \gamma\, V^\pi(s') \;\big|\; s,a\big] \qquad Q = \text{récompense} + \gamma V \text{ suivant}$$

Analogie – GPS dans une ville

G = temps total restant pour un trajet précis.
$V(s)$ = "à quel point cet endroit est bien situé ?" (centre-ville = V élevé).
$Q(s,a)$ = "si je suis ici ET que je prends cette rue, c'est bien ?"

Question type examen (Q3a/b/c, 3×1 pt)

"Donnez la définition de la Value Function V, de la Q-function Q, et du Gain G."

Réponses : les 3 formules encadrées ci-dessus avec $\mathbb{E}[G_t|\cdots]$.

4 S4. Discount Factor γ (1 pt)

Définition et rôle de γ

$\gamma \in [0,1]$ pondère l'importance des récompenses futures vs immédiates.

- $\gamma = 0 \;\rightarrow\;$ agent **myope** (ne regarde que la récompense immédiate)
- $\gamma \to 1 \;\rightarrow\;$ agent **prévoyant** (valorise autant le futur que le présent)
- Quand il n'y a pas d'état terminal, $\gamma < 1$ **garantit la convergence** du gain G

Analogie – Argent dans le temps

Préfères-tu 100 € aujourd'hui ou 100 € dans 1 an ? Évidemment aujourd'hui. $\gamma = 0.9$ signifie que 100 € dans 1 an ne valent que 90 € aujourd'hui. Plus γ est petit, plus l'agent est impatient.

Question type examen (Q4, 1 pt)

"Qu'est-ce que le discount factor γ et à quoi sert-il ?"

Réponse : définition $\gamma \in [0,1]$ + rôle (vision temporelle + convergence).

5 S5. Politique π (revient partout)

Types de politique

Déterministe : $a = \pi(s) \;\rightarrow\;$ une action précise par état.

Stochastique : $\pi(a|s) = P(A = a \mid S = s) \;\rightarrow\;$ distribution de probabilité.

$$\sum_a \pi(a|s) = 1 \quad \forall\, s$$

Politique optimale π^* : maximise $V(s)$ pour tout état.

$$\pi^*(s) = \arg\max_a Q^*(s, a)$$

Analogie – Conduite

Déterministe : au feu rouge, je freine TOUJOURS.
Stochastique : au feu orange, 70% je freine, 30% je passe.
Optimale : le conducteur parfait qui prend toujours la meilleure décision.

6 S6. Équation de Bellman (2+ pts)

Bellman Expectation (pour une politique π donnée)

$$\boxed{V^\pi(s) = \mathbb{E}[r_t + \gamma\, V^\pi(S_{t+1}) \mid S_t = s]}$$

Forme développée avec les probabilités :

$$V^\pi(s) = \sum_a \pi(a|s) \left[\sum_r r\, p(r|s, a) \;+\; \gamma \sum_{s'} p(s'|s, a)\, V^\pi(s') \right]$$

"Valeur d'un état = récompense immédiate + $\gamma\times$ valeur de l'état suivant."

Bellman Optimality (pour la politique optimale)

$$\boxed{V^*(s) = \max_a\; \mathbb{E}[r_t + \gamma\, V^*(S_{t+1}) \mid S_t = s,\; A_t = a]}$$

$$\boxed{Q^*(s, a) = \mathbb{E}\left[r + \gamma \max_{a'} Q^*(s', a') \;\middle|\; s, a \right]}$$

Analogie – Prix d'un appartement

Valeur de l'appart = loyer de ce mois (récompense immédiate) + $\gamma\times$ valeur de l'appart le mois prochain (valeur future). Pas besoin de calculer 50 ans de loyers.

Savoir l'appliquer – Exemple de calcul

Deux états s_1, s_2, une action, $\gamma = 0.9$:
- Depuis s_2 : reste en s_2, récompense 3 $\rightarrow$ $V(s_2) = 3 + 0.9 \cdot V(s_2)$ $\rightarrow$ $V(s_2) = 30$
- Depuis s_1 : va en s_2, récompense 5 $\rightarrow$ $V(s_1) = 5 + 0.9 \times 30 = 32$

Méthode : écrire l'équation de Bellman pour chaque état, résoudre le système linéaire.

Questions type examen (Q15, 1 pt + Q16, 1 pt)

Q15 : "Donnez l'équation de Bellman pour V et expliquez pourquoi elle est utile."
Réponse : formule de Bellman Expectation + "transforme un problème global en récursion locale".

Q16 : "Calculez la valeur manquante dans ce MDP (diagramme donné, $\gamma = 1$)."
Méthode : (1) Écrire Bellman pour le nœud manquant, (2) substituer les V connus, (3) résoudre le système si interdépendance entre nœuds.

7 S7. Pseudo-code d'un algorithme DP (3 pts)

Value Iteration (le plus simple – à connaître par cœur)

```
Entrée : MDP (S, A, P, R, γ)
Initialiser V(s) = 0 pour tout s ∈ S
Répéter jusqu'à convergence (max_s |V_new(s) − V_old(s)| < ε) :
   Pour chaque état s ∈ S :
      V(s) ← max_a [Σ_r r · p(r|s,a) + γ Σ_s' p(s'|s,a) · V(s')]
Extraire π*(s) = arg max_a [Σ_r r · p(r|s,a) + γ Σ_s' p(s'|s,a) · V(s')]
Retourner V*, π*
```

Policy Iteration (alternative)

```
Initialiser π(s) arbitrairement
Répéter jusqu'à stabilité de π :
   1. Policy Evaluation :
      Résoudre V^π(s) = Σ_r r · p(r|s,π(s)) + γ Σ_s' p(s'|s,π(s)) · V^π(s')
   2. Policy Improvement :
      π(s) ← arg max_a [Σ_r r · p(r|s,a) + γ Σ_s' p(s'|s,a) · V(s')]
```

Algorithme	Objectif	Équation
Policy Evaluation	Calculer V^π pour π fixé	Bellman Expectation
Policy Iteration	Trouver π^*	Bellman Expect. + Greedy
Value Iteration	Trouver V^* et π^*	Bellman Optimality

Analogie – Labyrinthe

Imagine un labyrinthe de 4 cases. Au départ, $V = 0$ partout. À chaque itération, on met à jour chaque case : "si je prends la meilleure direction, récompense + valeur de la case suivante". Après quelques tours, les valeurs se stabilisent et le chemin optimal apparaît.

Question type examen (Q17, 3 pts)

"Donnez le pseudo-code d'un algorithme de programmation dynamique."

Ce qu'on attend : pseudo-code complet de Value Iteration OU Policy Iteration, avec critère d'arrêt et extraction de π^*.

8 S8. Les 4 grandes dichotomies (4×1 pt)

Model-Free vs Model-Based

Model-Free	**Model-Based**
Apprend **directement** par interaction	Utilise un **modèle** de l'env. (P, R)
Pas besoin de connaître P ni R	Besoin de P et R (connus ou appris)
− Beaucoup de données	+ Meilleure sample efficiency
Ex : Q-Learning, SARSA, PPO	Ex : Value Iteration, MuZero

Value-Based vs Policy-Based

Value-Based	**Policy-Based**
Apprend $V(s)$ ou $Q(s, a)$	Apprend directement $\pi(a\|s)$
Politique **déduite** (greedy)	Pas de valeur explicite
Fonctionne bien en **discret**	Fonctionne aussi en **continu**
Ex : Q-Learning, DQN	Ex : REINFORCE, TRPO

Actor-Critic = les deux : politique (actor) + valeur (critic). Ex : A2C, PPO, SAC, TD3.

On-Policy vs Off-Policy

On-Policy	**Off-Policy**
Apprend la valeur de la politique **qu'il suit**	Apprend la valeur d'une **autre** politique
Données "fraîches" requises	Peut réutiliser d'anciennes données
Ex : SARSA, A2C, PPO	Ex : Q-Learning, DQN, SAC, TD3

Mnémotechnique : SARSA → $Q(s', a')$ (action réelle) = on-policy. Q-Learning → $\max_{a'} Q(s', a')$ = off-policy.

TD vs Monte Carlo

Temporal Difference (TD)	**Monte Carlo (MC)**
Met à jour à **chaque pas**	Met à jour en **fin d'épisode**
Bootstrapping (estimation → estimation)	Retour complet G_t (pas de bootstrap)
Biaisé, faible variance	**Non biaisé**, haute variance
Fonctionne en continu	Épisodes complets requis

Formules clés :
MC : $V(s) \leftarrow V(s) + \alpha\big[\mathbf{G_t} - V(s)\big]$
TD(0) : $V(s) \leftarrow V(s) + \alpha\big[\mathbf{r} + \gamma\mathbf{V}(\mathbf{s}') - V(s)\big]$

Analogies pour les 4 dichotomies

Model-Free/Based : Cuisiner par essai-erreur (free) vs suivre une recette (based).
Value/Policy : Noter chaque case sur 10 et aller à la meilleure (value) vs apprendre une stratégie directe (policy).
On/Off-Policy : Apprendre de tes propres copies (on) vs lire les copies des autres (off).
TD/MC : Recalculer ton temps de marathon à chaque km (TD) vs attendre la fin pour regarder le chrono (MC).

Questions type examen (Q6–Q9, 4×1 pt)

Q6 : "Model-Free vs Model-Based ?" – Q7 : "Value-Based vs Policy-Based ?"
Q8 : "On-Policy vs Off-Policy ?" – Q9 : "TD vs Monte Carlo ?"

Pour chacune : tableau 2 colonnes avec les différences clés + un exemple d'algorithme de chaque côté.

TIER A — TRÈS RENTABLE (∼10 pts, 20 min)

9 A1. Exploration vs Exploitation (1–2 pts)

Le dilemme et les stratégies

Exploration : essayer des actions inconnues pour découvrir de meilleures stratégies.
Exploitation : choisir les actions connues comme les meilleures.

ε-greedy (le plus demandé) :

$$\text{Avec proba } (1-\varepsilon) : a = \arg\max_a Q(s,a) \quad \text{(exploitation)}$$

$$\text{Avec proba } \varepsilon : a = \text{action aléatoire} \quad \text{(exploration)}$$

On peut faire décroître ε au fil du temps.

Autres stratégies : **Softmax/Boltzmann** ($P(a) \propto e^{Q(s,a)/\tau}$), **UCB** ($Q$ + bonus d'incertitude).

Analogie – Choisir un restaurant

Exploitation = retourner à ta pizzeria préférée. **Exploration** = tester un nouveau restaurant coréen.
ε-greedy : 90% pizzeria, 10% nouveau. Au fil des années, tu baisses le ε.

Question type examen (Q11, 1 pt)

"Expliquez le dilemme Exploration/Exploitation et donnez une stratégie."

Réponse : définir le dilemme + décrire ε-greedy avec avantages/inconvénients.

10 A2. Bootstrapping (1 pt)

Définition

Bootstrapping = mettre à jour une estimation en utilisant **une autre estimation** (pas la valeur réelle).

En TD : $V(s) \leftarrow V(s) + \alpha\big[r + \gamma \cdot \underbrace{V(s')}_{\text{estimation !}} - V(s)\big]$ ✓ bootstrapping

En MC : $V(s) \leftarrow V(s) + \alpha\big[\underbrace{G_t}_{\text{vrai retour}} - V(s)\big]$ × pas de bootstrapping

Conséquence : biais (on se fie à nos estimations) mais variance réduite.

Question type examen (Q12, 1 pt)

"Qu'est-ce que le bootstrapping ? Donnez un exemple."

Réponse : définition + formule TD montrant que $V(s')$ est une estimation.

11 A3. Actor-Critic (1 pt)

Architecture Actor-Critic

- **Actor (Acteur)** : apprend la politique π_θ → "quelle action prendre ?"
- **Critic (Critique)** : apprend V ou Q → "cette action est-elle bonne ?"

Fonction Avantage :

$$A(s, a) = Q(s, a) - V(s)$$

$A > 0 \rightarrow$ action meilleure que la moyenne → renforcer.
$A < 0 \rightarrow$ action pire que la moyenne → décourager.

Exemples : A2C, A3C, PPO, SAC, TD3.

Analogie – Tournage de film

Actor = l'acteur qui joue la scène. **Critic** = le réalisateur qui dit "C'était bien !" ou "On refait !". L'acteur s'améliore grâce aux retours du réalisateur.

Question type examen (Q10, 1 pt)

"Qu'est-ce qu'un algorithme Actor-Critic ?"

Réponse : décrire les 2 composants (actor = politique, critic = valeur) + fonction avantage.

12 A4. Replay Buffer (1 pt)

Définition et intérêt

Mémoire qui stocke les transitions (s, a, r, s') passées. On tire aléatoirement des mini-batches.

1. **Casse la corrélation temporelle** → stabilise l'apprentissage
2. **Réutilise les données** → meilleure sample efficiency
3. **Indispensable pour l'off-policy** (DQN, SAC, TD3)

PAS utilisé en on-policy (PPO, A2C).

Question type examen (Q13, 1 pt)

"Qu'est-ce qu'un Replay Buffer et pourquoi en utiliser un ?"

Réponse : définition + 3 raisons (corrélation, réutilisation, off-policy).

13 A5. Fonction de récompense efficace (1 pt)

Principes d'une bonne récompense

- Récompenses **positives** pour le comportement souhaité
- Récompenses **négatives** (pénalités) pour les comportements indésirables
- Doit être **fréquente et informative** (pas juste à la fin → problème de récompense sparse)

Exemple – Drone volant : $+1$/s en vol stable, $+10$ destination, -100 crash, -0.1/s (encourage vitesse).

Question type examen (Q5, 1 pt)

"Comment définir une récompense efficace ? Donnez un exemple concret."

Réponse : 3 principes + un exemple concret avec récompenses positives, négatives et fréquentes.

14 A6. Q-Learning et SARSA (3+ pts)

SARSA – On-Policy TD Control

$$\boxed{Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[r_t + \gamma\, Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big]}$$

A_{t+1} = action **réellement prise** (ε-greedy) $\Rightarrow$ **on-policy**.

Q-Learning – Off-Policy TD Control

$$\boxed{Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[r_t + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\Big]}$$

$\max_{a'}$ = meilleure action **théorique** $\Rightarrow$ **off-policy**.

	SARSA	**Q-Learning**
Mise à jour	$r + \gamma \cdot Q(s', a')$	$r + \gamma \cdot \max_{a'} Q(s', a')$
Type	On-policy	Off-policy
Équation	Bellman Expectation	Bellman **Optimality**
Comportement	Prudent	Agressif (vise l'optimal)

Analogie – Traverser une rivière

SARSA (prudent) : tu poses le pied sur chaque pierre et tu notes si elle glisse → chemin sûr.
Q-Learning (agressif) : tu imagines toujours la meilleure pierre → chemin optimal théorique, mais tu peux te mouiller pendant l'apprentissage.

Question type examen (Q21, ∼3 pts)

"Identifiez l'algorithme du pseudo-code suivant. Est-il value-based ou policy-based ? TD ou MC ?"

Indice : si on voit $\max_{a'}$ dans la mise à jour → Q-Learning (value-based, TD).
Si on voit a' pris par ε-greedy → SARSA (value-based, TD aussi).

15 A7–A8. Programmation Dynamique : principe et limites (2 pts)

Principe de la DP

Résoudre un problème complexe en le **décomposant en sous-problèmes** qui se chevauchent. On résout chaque sous-problème **une seule fois**, on **stocke** le résultat, on le **réutilise**.

En RL : on exploite la structure récursive de Bellman → $V(s) = r + \gamma V(s')$.

Avantages et Inconvénients de la DP

Avantages	**Inconvénients**
Convergence **garantie**	Nécessite le **modèle complet** (P, R)
Solution **exacte**	**Coûteux** pour grands espaces (curse of dim.)
Base théorique solide	Planification, pas d'exploration

Questions type examen (Q14, 1 pt + Q18, 1 pt)

Q14 : "Principe de la DP au-delà du RL." → Décomposition + stockage + réutilisation.
Q18 : "Avantages et inconvénients de la DP." → Tableau ci-dessus.

TIER B — IMPORTANT (∼10 pts, 30 min)

16 B1. Cas continu – 4 concepts (4 pts)

Le problème du continu

Exemple : bras robotique → angles $\in \mathbb{R}^n$ (états continus), couples moteurs $\in \mathbb{R}^m$ (actions continues).

Défis :

1. Impossible de **tabuler** $V(s)$ ou $Q(s,a)$ (espaces infinis)
2. Nécessite de la **généralisation** (estimer la valeur d'états jamais visités)
3. **Exploration** beaucoup plus difficile (espace infini)
4. Les algorithmes tabulaires ne fonctionnent plus directement

Solution : Approximation de fonction

On paramétrise $V_\theta(s)$, $Q_\theta(s,a)$ ou $\pi_\theta(a|s)$ par un **réseau de neurones**.
Le réseau apprend à **généraliser** depuis les états visités vers les états non visités.
On remplace la table $Q[s][a]$ par une fonction $Q_\theta(s,a)$.

Algorithme pour le continu : PPO

PPO (Proximal Policy Optimization) : Actor-Critic, on-policy.
La politique π_θ sort une distribution **continue** (ex : gaussienne).
Clipped surrogate objective pour la stabilité. Très populaire (RLHF, ChatGPT).

Autres : **SAC** (off-policy, max entropie), **TD3** (off-policy, déterministe), **DDPG**.

Questions type examen (Q22–Q25, 4×1 pt)

Q22 : "Exemple concret de RL en continu." → Bras robotique, voiture autonome.
Q23 : "Principaux défis du cas continu." → 4 défis ci-dessus.
Q24 : "Outil pour adapter les méthodes discrètes." → Approximation de fonction (réseau de neurones).
Q25 : "Un algorithme pour le continu." → PPO (ou SAC, TD3).

17 B2. Deep RL – DQN (1 pt)

Architecture DQN

Entrée : état s (ex : pixels Atari).
Sortie : vecteur de Q-values $[Q(s,a_1), Q(s,a_2), \ldots, Q(s,a_n)]$.
Action : $\arg\max$ sur la sortie.

Innovations clés :

1. **Replay buffer** : stocke les transitions, mini-batches aléatoires
2. **Target network** : second réseau "gelé" pour calculer la cible → stabilise l'apprentissage

Question type examen (Q28, 1 pt)

"Décrivez l'architecture DQN : entrées et sorties."

Réponse : état s en entrée → réseau de neurones → vecteur de Q-values en sortie → argmax → action.

18 B3. Policy Gradient Theorem (1–2 pts)

Le théorème

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t\right]}$$

Ce que ça signifie : on peut estimer le gradient de la performance **sans connaître le modèle**, uniquement en observant des trajectoires.

Intuition : trajectoire avec bon retour → augmenter la proba des actions prises.
Trajectoire avec mauvais retour → diminuer la proba.

Analogie – Lancer franc au basket

Tu lances 100 fois, tu notes ta technique. Lancers réussis → répéter la technique. Lancers ratés → éviter. Après 1000 lancers, tu converges vers la meilleure technique.

Question type examen (Q29a, 1 pt)

"Sur quel théorème repose l'algorithme ? Expliquez."

Réponse : Policy Gradient Theorem + formule + "permet d'optimiser sans modèle".

19 B4. Fonction objectif $J(\theta)$ (1 pt)

Définition

$$\boxed{J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]}$$

Espérance du retour cumulé sur les trajectoires τ générées par π_θ.
Optimisation : montée de gradient → $\theta \leftarrow \theta + \alpha \cdot \nabla_\theta J(\theta)$.

Question type examen (Q27, 1 pt)

"Quelle fonction optimise-t-on quand la politique est paramétrée par θ ?"

Réponse : $J(\theta) = \mathbb{E}[R(\tau)]$ + gradient ascent.

20 B5. REINFORCE (1 pt)

REINFORCE = Monte-Carlo Policy Gradient

```
Pour chaque épisode :
    Générer τ = (s_0, a_0, r_0, s_1, a_1, r_1, ...) en suivant π_θ
    Pour chaque pas t :
        G_t = Σ_{k=0}^{T-t} γ^k r_{t+k}      (retour MC)
        θ ← θ + α · ∇_θ log π_θ(a_t|s_t) · G_t
```

Caractéristiques : policy-based, model-free, non biaisé mais **haute variance**.
Amélioration : ajouter une baseline $V(s)$ → A2C (Actor-Critic).

Question type examen (Q29c/d, 1–2 pts)

"Limitation concrète de l'algorithme et amélioration possible ?"

Réponse : haute variance du retour MC → convergence lente. Amélioration : baseline $V(s)$ → avantage $A(s,a) = G_t - V(s)$ → Actor-Critic (A2C).

21 B6. Intérêt des réseaux de neurones en RL (1 pt)

Pourquoi les réseaux de neurones ?

Approximateurs de fonctions universels pour :

1. Représenter V, Q ou π dans des espaces **très grands ou continus**
2. **Généraliser** à des états jamais visités
3. Traiter des entrées **complexes** (images, signaux, texte)

Sans eux → limité aux tables $Q[s][a]$ → petits environnements discrets uniquement.

Question type examen (Q26, 1 pt)

"Quel est l'intérêt principal des réseaux de neurones en RL ?"

Réponse : approximation de fonctions dans des espaces grands/continus + généralisation.

TIER C — BONUS / SI TEMPS RESTANT (∼6 pts + bonus)

22 C1. PPO – 3 caractéristiques (1 pt)

PPO (Proximal Policy Optimization)

1. **Actor-Critic** : acteur π_θ + critique V_ϕ
2. **Clipped surrogate objective** : limite les mises à jour → stabilité (trust region de TRPO)
3. **Bonus d'entropie** : encourage l'exploration, évite la convergence prématurée

On-policy. Utilisé pour RLHF (ChatGPT).

Question type examen (Q30, 1 pt)

"Donnez 3 caractéristiques de PPO."

Réponse : les 3 points ci-dessus.

23 C2. Model-Based RL (2 pts + 1 bonus)

Motivation et difficulté

Motivation : **sample efficiency** → avec un modèle, l'agent peut **planifier et simuler** sans interagir réellement. Réduit le nombre d'interactions nécessaires.

Difficulté principale : apprendre un modèle fidèle est très dur. Les erreurs **s'accumulent** au fil des pas de planification (compound error).

MuZero – Exemple d'algorithme Model-Based

- Apprend un modèle interne (représentation latente) prédisant : valeur, politique, récompense
- Utilise **MCTS** pour planifier
- Appliqué à **Atari** et **Go**, **sans connaître les règles**

Lignée : AlphaGo (règles + données) → AlphaZero (règles) → MuZero (rien) → EfficientZero.

Questions type examen (Q31–Q33, 2 pts + 1 bonus)

Q31 : "Motivation principale du Model-Based." → Sample efficiency.
Q32 : "Principale difficulté." → Compound error du modèle appris.
Q33 (bonus) : "Exemple d'algorithme + application." → MuZero + Atari/Go.

24 C3. Preuve de l'équation de Bellman (3 pts bonus)

Preuve

Par définition : $G = r_0 + \gamma \cdot G_1$ où G_1 est le gain à partir de s_1.

$V^\pi(s) = \mathbb{E}[G \mid s_0 = s] = \mathbb{E}[r_0 + \gamma G_1 \mid s_0 = s] = \mathbb{E}[r_0 \mid s_0 = s] + \gamma \cdot \mathbb{E}[G_1 \mid s_0 = s]$

Par la **loi de l'espérance totale** (Fubini) :
$\mathbb{E}[G_1 \mid s_0 = s] = \mathbb{E}\big[\, \mathbb{E}[G_1 \mid s_0 = s, s_1] \,\big|\, s_0 = s\big]$

Par la **propriété de Markov** :
$\mathbb{E}[G_1 \mid s_0 = s, s_1] = \mathbb{E}[G_1 \mid s_1] = V^\pi(s_1)$

Conclusion :

$$\boxed{V^\pi(s) = \mathbb{E}[r_0 + \gamma \cdot V^\pi(s_1) \mid s_0 = s] \qquad \blacksquare}$$

Convergence : l'opérateur de Bellman est **contractant** ($\gamma < 1$) $\rightarrow$ **théorème de Banach** (point fixe).

Question type examen (Q19, 3 pts bonus)

"Prouvez l'équation de Bellman."

Réponse : développer $G = r_0 + \gamma G_1$, appliquer espérance totale + Markov.

25 C4. Algorithmes Deep RL avancés (culture)

TD3, SAC, RLHF, GRPO

Algorithme	Caractéristiques clés
TD3	Off-policy, actor-critic, continu. Double Q-learning clippé + mises à jour retardées + lissage cible.
SAC	Off-policy, actor-critic, continu. Maximum entropy RL. Deux critiques.
RLHF	Aligner les LLMs via PPO + reward model entraîné sur préférences humaines.
GRPO	DeepSeek. Pas de critic, baseline = moyenne du groupe. KL divergence. Adapté aux LLMs.

26 C5. MCTS – Monte Carlo Tree Search (culture)

Les 4 étapes de MCTS

1. **Selection** : descendre dans l'arbre vers les branches prometteuses
2. **Expansion** : ajouter un nouveau nœud (état)
3. **Simulation** : simuler une partie aléatoire depuis ce nœud
4. **Backpropagation** : remonter le résultat pour mettre à jour les valeurs

Utilisé dans AlphaGo/AlphaZero/MuZero.

RÉSUMÉ ULTRA-CONDENSÉ — La veille de l'examen

Les 10 choses à savoir ABSOLUMENT

1. **Schéma RL** : Agent $\leftrightarrow$ Environnement (S, A, R) + 5 définitions
2. **MDP** $= (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ + propriété de Markov
3. G, V, Q : $G = \sum \gamma^k r, \quad V = \mathbb{E}[G|s], \quad Q = \mathbb{E}[G|s, a]$
4. γ : pondère futur vs présent, assure convergence
5. **Bellman** : $V(s) = \mathbb{E}[r + \gamma V(s')|s]$ et savoir l'appliquer
6. **Value Iteration ou Policy Iteration** : pseudo-code complet
7. **TD vs MC** : bootstrapping vs retour complet, biaisé vs non biaisé
8. **Q-Learning vs SARSA** : max vs action réelle, off vs on policy
9. **Policy Gradient Theorem** : $\nabla J = \mathbb{E}[\nabla \log \pi \cdot G_t]$
10. **PPO** : actor-critic + clipping + entropie

Les 6 distinctions à réciter

- Model-Free vs Model-Based
- Value-Based vs Policy-Based
- On-Policy vs Off-Policy
- TD vs Monte Carlo
- Exploration vs Exploitation
- Prediction vs Control

Les algorithmes à citer par catégorie

Catégorie	Algorithmes
DP (model-based)	Policy Evaluation, Policy Iteration, Value Iteration
TD model-free	SARSA (on), Q-Learning (off), Expected SARSA
MC model-free	First-Visit MC, Every-Visit MC
Policy Gradient	REINFORCE
Actor-Critic	A2C, A3C, PPO, TRPO, SAC, TD3, DDPG
Deep RL	DQN (+ replay buffer + target network)
Model-Based	AlphaGo, AlphaZero, MuZero, EfficientZero
LLM alignment	RLHF (PPO), GRPO (DeepSeek)

Catégorie	Questions	Points	Difficulté
Définitions (par cœur)	Q1–5, Q14	8 pts	Facile
Distinctions conceptuelles	Q6–13	8 pts	Facile
Bellman	Q15–16	2 pts	Moyen
Pseudo-code DP	Q17	3 pts	Moyen
Avantages/inconvénients	Q18	1 pt	Facile
Model-free (identification)	Q20–21	4 pts	Moyen
Cas continu	Q22–25	4 pts	Facile
Deep RL	Q26–30	7 pts	Moyen
Model-Based	Q31–32	2 pts	Facile
Bonus	Q19, Q29d, Q33	4 pts	Difficile
TOTAL		**40 + 4 bonus**	

Stratégie d'examen

Les **16 premiers points** (définitions + distinctions) sont du par-cœur pur → 30 min.
Les **6 pts** de Bellman + DP demandent de comprendre la récursion.
Le reste (continu, deep RL, model-based) = compréhension générale.
Les bonus nécessitent un vrai travail de fond (preuve, identification fine).